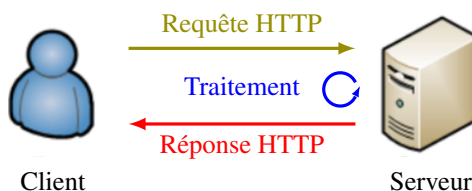


## 1 Rappels sur le modèle client/serveur

On a vu dans le cours précédent que la consultation de pages sur un site fonctionne sur une architecture client/serveur :

- ★ un internaute connecté au réseau via son ordinateur et un navigateur web est le client ;
- ★ le serveur correspond aux ordinateurs contenant les applications qui fournissent les pages demandées ;
- ★ le client et le serveur communiquent entre eux en utilisant un protocole de communication (HTTP).



On distingue deux grandes familles de sites web :

### Définition 1.1 (Interactif vs dynamique)

- ★ Les sites web dits « interactifs » sont les sites web pour lesquels les modifications des pages ont lieu *uniquement* du côté client.
- ★ Les sites web dits « dynamiques » sont les sites web pour lesquels des modifications de pages ont lieu du côté serveur.

Dans le cas d'un site web dynamique, des données sont envoyées, via la requête, à un fichier cible du serveur qui les traite et qui renvoie en retour une page web dont le contenu dépend des données reçues.

**Exemple 1.2** Considérons la requête suivante :

`http://127.0.0.1:80/HelloWorld/connexion.php?nom=Durand&prenom=Jean&sexe=0`

Celle-ci se décompose en quatre parties :

1. `http://` indique le protocole utilisé pour la communication entre le navigateur et le serveur qui héberge le fichier cible utilisé ;
2. `127.0.0.1:80` indique l'adresse de la machine qui héberge la ressource<sup>1</sup> ;
3. `/HelloWorld/connexion.php` indique le chemin vers le fichier cible ;
4. `?nom=Durand&prenom=Jean&sexe=0` indique l'ensemble des données transmises au fichier cible (il y en a trois).

Le fichier cible `connexion.php` reçoit les données du point 4 et les traite pour générer une page web en retour. Supposons par exemple que le code de ce fichier est le suivant :

```

1  <!DOCTYPE html>
2  <html lang="fr">
3
4      <head>
5          <meta charset="UTF-8" />
6          <title>Connexion</title>
7      </head>
8
9      <body>
10         <h1>Connexion réussie !!!</h1>
11
12     <?php
13         $N = $_GET['nom'];
14         $P = $_GET['prenom'];
15         $S = $_GET['sexe'];
16
17         if($S == '0')
18         {
19             $S = "monsieur";
20         }
21         else
22         {
23             $S = "madame";
24         }
25
26         echo "<p>Bonjour $S $P $N. Comment allez-vous ?</p>";
27     ?>
28
29
30     </body>
31 </html>
  
```

On observe qu'il y a deux langages utilisés :

- ★ le langage HTML aux lignes 1-10 et 30-31 qui permet de créer une page web ;
- ★ le langage PHP aux lignes 12-27 qui est un langage de programmation permettant de gérer des variables et de s'adapter ainsi aux données reçues.

1. On utilise ici un serveur local (type EasyPHP, Wamp, Mamp, etc.) situé à l'adresse `127.0.0.1` (cette adresse est celle de l'interface logique `localhost` de l'ordinateur local) et qui écoute les requêtes sur le port 80 de l'ordinateur.

La récupération des données transmises lors de la requête s'effectue aux lignes 13-15. Un traitement est ensuite appliqué aux lignes 17-24. Enfin, la page web est renvoyée à la ligne 26. On observe en particulier dans cette ligne la génération dynamique de la page puisque ce sont les variables qui sont affichées à l'aide de l'instruction `echo`. On observe également que cette instruction renvoie du code HTML.

**Remarque 1.3** Pour générer dynamiquement une page web, il est nécessaire de disposer d'un langage de programmation exécutable par le serveur. Le fichier cible contient donc nécessairement un script qui peut être écrit dans différents types de langage (Python, PHP, etc.). Dans ce cours, nous faisons le choix d'utiliser le langage PHP puisque c'est celui qui est actuellement le plus couramment utilisé.

## 2 Qu'est-ce que le langage PHP ?

Le principe général du langage PHP (= PHP :Hypertext Preprocessor; acronyme récursif) est de créer des sites web dynamiques, c'est-à-dire des sites web qui puissent afficher du contenu changeant sans l'intervention d'un développeur pour en modifier le code. C'est un langage de programmation libre créé en 1994 par Rasmus Lerdorf de la famille des langage impératif orienté objet, mais qui peut aussi fonctionner comme n'importe quel langage interprété de façon locale. Il est actuellement développé par *The PHP Group*. La version la plus récente est la version 7 de janvier 2019.

## 3 Insertion d'un code PHP

- ★ Un code source PHP peut être inséré n'importe où dans un code HTML et doit être encadré par la balise ouvrante `<?php` et la balise fermante `?>` (voir exemple 1.2).
- ★ Un fichier contenant un code PHP doit obligatoirement être sauvegardé avec l'extension `.php`.

## 4 Syntaxes élémentaires en PHP

### 4.1 Instructions, variables, commentaires et sorties

- ★ **Instructions et variables.**
  - Une **instruction** se termine toujours par un point-virgule ;
  - Les **variables** sont définies avec le préfixe `$` et une affectation.

**Exemple 4.1** L'instruction `$x = 2;` permet de définir une variable `$x` et de lui affecter la valeur 2.

- ★ **Commentaires.**
  - Les **commentaires sur une ligne** commencent par `//`
  - Les **commentaires sur plusieurs lignes** sont encadrés par `/*` et `*/`
- ★ **Pour afficher une chaîne de caractères sur une page web**, on utilise les instructions `echo '...'` ou `echo "..."`.
  - Si une variable est écrite entre les guillemets doubles, elle est automatiquement remplacée par sa valeur (sauf si elle correspond à une donnée d'un tableau ou à une valeur renvoyée par une fonction).
  - Si une variable est écrite entre les guillemets simples, elle n'est pas évaluée.
  - Tant que l'on n'a pas refermé le guillemet double `"`, on peut passer à la ligne pour écrire sur plusieurs lignes.
  - On peut utiliser n'importe quelle balise HTML; le navigateur du client interprétera le code à réception de la page.

**Exemple 4.2** Le code PHP

```
$N = 'Durand';
$P = 'Jean';
$M = 'monsieur';
echo "<p>Bonjour $M $P $N. Comment allez-vous ?</p>";
```

permet d'afficher le texte suivant sur la page web :

```
Bonjour monsieur Jean Durand. Comment allez-vous ?
```

## 4.2 Les tableaux

Les tableaux PHP sont des données structurées analogues aux dictionnaires Python. Ils sont appelés *tableaux associatifs* et sont donc définis par des paires (clé; valeur). Plus précisément, pour définir un tableau \$tab, on utilise la syntaxe suivante :

```
$tab = array(cle1 => val1, cle2 => val2, ...)
```

Pour obtenir la valeur associée à une clé cle, il suffit alors de saisir l'instruction \$tab[cle].

**Exemple 4.3** Considérons le tableau \$fruits = array('oranges' => 40, 'poires' => 5). Alors, l'instruction \$fruits['poires'] renvoie la valeur 5.

**Remarque 4.4** On peut aussi définir un tableau avec la syntaxe suivante :

```
$tab = array(val1, val2, ...)
```

Dans ce cas, les clés sont implicites et sont les nombres entiers 0, 1, etc. On parle alors de tableaux *indexé numériquement* (mais ce sont encore des tableaux associatifs !) et ils sont analogues aux listes Python.

## 4.3 Structure d'embranchement si ... alors ... sinon

| Syntaxe                                                                                                                            | Exemple                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| <pre>if (condition) {     instruction 1;     ...     instruction n; } else {     instruction 1;     ...     instruction m; }</pre> | <p>Affichage du signe d'une variable \$x :</p> <pre>if (\$x&lt;=0) {     echo 'négatif'; } else {     echo 'positif'; }</pre> |

## 4.4 Structure de boucle itérative pour

| Syntaxe                                                                                            | Exemple                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>for(init; condition ; incrémentation) {     instruction 1;     ...     instruction n; }</pre> | <p>Affichage des éléments d'un tableau par index :</p> <pre>\$tab = [3,2,1,'Partez !!!']; for(\$i=0; \$i&lt;4; \$i++) {     echo \$tab[\$i]. '&lt;br/&gt;'; }</pre> |

**Remarque 4.5** L'opérateur . permet de concaténer deux chaînes de caractères.

## 4.5 Structure de boucle conditionnelle tant que

| Syntaxe                                                                       | Exemple                                                                                                                                                                      |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>while(condition) {     instruction 1;     ...     instruction n; }</pre> | <p>Affichage des éléments d'un tableau par index :</p> <pre>\$tab = [3,2,1,'Partez !!!']; \$i=0; while(\$i&lt;4) {     echo \$tab[\$i]. '&lt;br/&gt;';     \$i += 1; }</pre> |

## 5 Les formulaires HTML

Un formulaire représente une section d'un document HTML contenant des contrôles interactifs permettant à un utilisateur d'envoyer des données à un serveur web.

### 5.1 Insertion d'un formulaire dans une page web

Pour insérer un formulaire dans une page HTML, on utilise la balise `<form> . . . </form>`. Il s'agit de la balise principale du formulaire : elle en indique le début et la fin. Cette balise admet deux attributs obligatoires :

- ★ `action` : indique le fichier cible pour l'envoi des données ;
- ★ `method` : indique le type d'envoi des données (GET ou POST).

**Exemple 5.1** Pour créer un formulaire envoyant des données en méthode GET au fichier `connexion.php`, on utilise le code suivant :

```
<form method='GET' action='connexion.php'>
    #saisie du formulaire
</form>
```

**Remarque 5.2** La méthode GET envoie au serveur les valeurs des champs du formulaire dans la barre d'URL avec le symbole ? comme séparateur (c'est la méthode utilisée dans l'exemple 1.2). En revanche, la méthode POST envoie directement les données du formulaire via le réseau au serveur sans passer par l'URL. On utilise en général cette méthode lorsque les champs des formulaires contiennent des caractères spéciaux ou lorsque les données sont nombreuses. Toutefois, aucune de ces méthodes n'est sécurisée. Il faut donc faire des contrôles sur le serveur.

### 5.2 Les zones de saisie

Les zones de saisie d'un formulaire sont généralement définies à l'aide de la balise générique `<input/>`. Les possibilités d'interactivité du client sur ces zones dépendent fortement de leur type (saisie de texte, cases à cocher, etc.).

Pour les différencier, cette balise admet un attribut `type` pouvant prendre diverses valeurs, par exemple :

- ★ `'text'` : permet de définir une zone de saisie de texte ;
- ★ `'email'` : permet de définir une zone de saisie d'adresse mail (des contrôles automatiques sont effectués pour le format) ;
- ★ `'password'` : permet de définir une zone de saisie de mot de passe (caractères cachés) ;
- ★ `'file'` : permet de définir une zone de téléversement de fichier ;
- ★ `'number'` : permet de définir une zone de saisie numérique ;
- ★ `'range'` : permet de définir une zone de saisie de numérique sous forme de curseur ;
- ★ `'radio'` : permet de définir un bouton radio (élément à cocher) ;
- ★ `'checkbox'` : permet de définir une case à cocher.

**Remarque 5.3** La différence entre les cases à cocher et les boutons radio est que le choix est unique dans les boutons radio.

Une fois le type de zone choisi, la balise `<input/>` admet un ou deux attributs obligatoires supplémentaires :

- ★ `name` : permet de définir le nom de la zone (valable pour tous les types) ;
- ★ `value` : permet de définir la valeur de la zone si elle est sélectionnée (valable pour les boutons radio et les cases à cocher).

Ces deux attributs sont fondamentaux car ce sont eux qui sont envoyés au fichier cible du formulaire.

**Exemple 5.4** Deux exemples de création de zones de saisie :

- ★ une zone de saisie de texte : `<input type='text' name='prenom' />`
- ★ un bouton radio : `<input type='radio' name='sexe' value='0' />`

En plus des attributs obligatoires, chaque type de zone admet plusieurs attributs optionnels. Par exemple :

- ★ `id` : permet de définir l'identifiant de la zone (valable pour tous les types) ;
- ★ `class` : permet de définir la classe de la zone (valable pour tous les types) ;
- ★ `placeholder` : permet de donner une indication sur le contenu de la zone (valable uniquement pour les saisies de texte) ;
- ★ `checked` : permet de cocher une option par défaut (valable uniquement pour les boutons radio et les cases à cocher) ;
- ★ `required` : permet de rendre une zone obligatoire (non valable pour les boutons radio et les cases à cocher).

Pour plus de détails sur les attributs optionnels des zones de saisie, on consultera le document **Introduction aux formulaires HTML5** ou internet.

◁ Une autre zone de saisie importante dans les formulaires est la *liste déroulante*. Celle-ci est encadrée par la balise ouvrante `<select>` et la balise fermante `</select>`. Elle se comporte comme une sélection d'option dans une liste, chaque option étant définie à l'aide de la balise `<option>...</option>`. A nouveau, plusieurs attributs sont obligatoires :

- ★ `name` (balise `<select>`) : permet de définir le nom de la liste ;
- ★ `value` (balise `<option>`) : permet de définir la valeur de l'option si elle est sélectionnée.

**Exemple 5.5** Un exemple de création de menu déroulant :

```
<select name='pays' id='pays'>
  <option value='allemagne'>Allemagne</option>
  <option value='france' selected>France</option>
  <option value='espagne'>Espagne</option>
  <option value='italie'>Italie</option>
  <option value='royaume-uni'>Royaume-Uni</option>
  <option value='canada'>Canada</option>
  <option value='etats-unis'>États-Unis</option>
  <option value='chine'>Chine</option>
  <option value='japon'>Japon</option>
</select>
```

Dans cet exemple, l'attribut `id` de la balise `<select>` et les attributs `selected` et `disabled` de la balise `<option>` sont optionnels. L'attribut `selected` est l'analogue de `checked` pour les listes et l'attribut `disabled` permet de mettre une option en lecture seule (et donc non sélectionnable).

### 5.3 Etiquetage des éléments d'un formulaire

Pour que le visiteur d'une page web contenant un formulaire sache à quoi correspondent les différentes zones affichées, on leur attribue un libellé avec la balise `<label>...</label>` que l'on associe à la zone correspondante à l'aide de l'attribut `for`. Cet attribut est associé à l'identifiant `id` de la zone et doit, bien sûr, avoir le même nom que lui.

**Exemple 5.6** Pour créer un label `Nom` : à une zone dont l'identifiant est `N`, on tape les instructions suivantes :

```
<label for='N'> Nom : </label>
<input type='text' name='nom' id='N' />
```

**Remarque 5.7** Le texte du label n'est pas seulement associé visuellement au champ, il lui est techniquement associé : lorsque l'utilisateur a le focus sur le champ, un lecteur d'écran pourra énoncer le contenu du label et permettre à l'utilisateur de disposer d'un meilleur contexte. On peut aussi cliquer sur le label pour avoir le focus sur le champ (intéressant si on utilise de petits appareils comme des téléphones portables).

### 5.4 Les boutons de contrôles

Pour finaliser totalement le formulaire, il est nécessaire de disposer de boutons de contrôles afin de réinitialiser le formulaire en cas d'erreur de saisie ou de soumettre le formulaire une fois celui-ci correctement rempli.

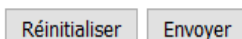
- ★ `<input type='reset' />` : cette balise permet de créer un bouton qui remet à zéro le formulaire.
- ★ `<input type='submit' />` : cette balise permet de créer un bouton qui envoie les données du formulaire au script définie par l'argument `action` de la balise `<form>`.

Ces deux boutons admettent l'argument `value` qui permet de changer le texte affiché à l'intérieur des boutons.

**Exemple 5.8** Un exemple de boutons `reset` et `submit` :

```
<input type='reset' value='Réinitialiser' />
<input type='submit' value='Envoyer' />
```

On obtient à l'écran le rendu suivant :



## 5.5 CSS et JavaScript

- ★ Tout élément d'un formulaire peut être modifié visuellement avec du CSS.
- ★ Tout élément d'un formulaire peut être modifié structurellement avec du JavaScript en agissant sur le contenu textuel ou sur les attributs (obligatoires ou optionnels).
- ★ La balise `<form>` admet l'attribut optionnel événementiel `onsubmit` qui permet de déclencher un événement JavaScript lors du clic sur le bouton `submit` du formulaire.

## 5.6 Gestion par le fichier PHP cible

1. Une fois rempli et validé avec le bouton `submit`, les données du formulaire sont envoyées au fichier cible défini par l'attribut `action` avec la méthode d'envoi définie par l'attribut `method`. Plus précisément, pour chaque zone du formulaire, les données envoyées sont structurées sous la forme d'une paire (clé; valeur), où la clé correspond à la valeur de l'attribut `name` et où la valeur correspond aux valeurs associées à la zone (contenu textuel pour les zones de texte, valeur des attributs `value`, etc.).
2. A réception par le fichier PHP cible, les données sont affectées à un tableau associatif de nom `$_GET` ou `$_POST` suivant la méthode d'envoi. Un tel tableau est créé automatiquement par le serveur.
3. Pour accéder à une valeur précise du formulaire, on utilise la syntaxe du tableau associatif.

**Exemple 5.9** Considérons le code HTML suivant :

```
1  <!DOCTYPE html>
2  <html lang="fr">
3
4      <head>
5          <meta charset="UTF-8" />
6          <title>Hello World</title>
7      </head>
8
9      <body>
10         <h1>Remplissez le formulaire de connexion ci-dessous</h1>
11         <form method="GET" action="connexion.php" onsubmit="return confirm('Confirmez votre choix ?');">
12             <table>
13                 <tr>
14                     <td><label for="N"> Nom : </label></td>
15                     <td><input type="text" name="nom" id="N" placeholder="Ex : Dupont" required /></td>
16                 </tr>
17                 <tr>
18                     <td><label for="P"> Prénom : </label></td>
19                     <td><input type="text" name="prenom" id="P" placeholder="Ex : Jean" required /></td>
20                 </tr>
21                 <tr>
22                     <td>Sexe :</td>
23                     <td>
24                         <input type="radio" name="sexe" id="M" value="0" checked />
25                         <label for="M"> masculin </label>
26                         <input type="radio" name="sexe" id="F" value="1" />
27                         <label for="F"> féminin </label>
28                     </td>
29                 </tr>
30             </table>
31             <input type="reset" value="Réinitialiser" />
32             <input type="submit" value="Valider" />
33         </form>
34     </body>
35 </html>
```

On suppose que l'on a saisi Durand dans la première zone de texte, Jean dans la seconde zone de texte et que l'on a coché le premier bouton radio. Une fois cliqué sur le bouton Valider, les données sont envoyées avec la méthode GET au fichier `connexion.php` sous la forme `('nom'; 'Durand')`, `('prenom'; 'Jean')` et `('sexe'; '0')`. A réception, on récupère les données saisies par l'utilisateur avec les variables `$_GET['nom']`, `$_GET['prenom']` et `$_GET['sexe']` (voir lignes 13-15 de l'exemple 1.2) qui renvoient ici 'Durand', 'Jean' et '0'.

**Remarque 5.10** Bien sûr, si l'utilisateur change les valeurs des données dans le formulaire, le code PHP n'est lui pas modifié car il utilise des variables intrinsèques au formulaire (les noms des zones). D'où le dynamisme des pages : c'est le *même* code PHP qui permet de générer des pages web *différentes* suivant les données saisies par l'utilisateur.